

# MELSS AOI-Pipeline

*PCB Automated Optical Inspection — Technical Reference*

---

## Contents

|          |                                                                    |           |
|----------|--------------------------------------------------------------------|-----------|
| <b>1</b> | <b>What This Project Does</b>                                      | <b>3</b>  |
| <b>2</b> | <b>Repository Layout</b>                                           | <b>3</b>  |
| <b>3</b> | <b>Setup</b>                                                       | <b>5</b>  |
| 3.1      | Python Version . . . . .                                           | 5         |
| 3.2      | Install All Dependencies . . . . .                                 | 5         |
| 3.3      | Model Weights . . . . .                                            | 5         |
| <b>4</b> | <b>The 25 Component Classes</b>                                    | <b>5</b>  |
| <b>5</b> | <b>File Reference — src/</b>                                       | <b>6</b>  |
| 5.1      | pcb_aoi.py — Interactive CLI Inspector . . . . .                   | 6         |
| 5.1.1    | What it does . . . . .                                             | 6         |
| 5.1.2    | Alignment algorithm . . . . .                                      | 6         |
| 5.1.3    | Comparison algorithm . . . . .                                     | 6         |
| 5.1.4    | Default thresholds . . . . .                                       | 6         |
| 5.1.5    | Three run modes . . . . .                                          | 6         |
| 5.1.6    | Outputs . . . . .                                                  | 7         |
| 5.1.7    | Usage . . . . .                                                    | 7         |
| 5.2      | generate_defect.py — Synthetic Defect Generator . . . . .          | 7         |
| 5.2.1    | What it does . . . . .                                             | 7         |
| 5.2.2    | The four injectors . . . . .                                       | 7         |
| 5.2.3    | Defect mix and naming . . . . .                                    | 8         |
| 5.2.4    | Outputs . . . . .                                                  | 8         |
| 5.2.5    | Usage . . . . .                                                    | 8         |
| 5.3      | aoi_evaluator.py — Accuracy Evaluator v3.9 . . . . .               | 9         |
| 5.3.1    | What it does . . . . .                                             | 9         |
| 5.3.2    | AOI inference zones . . . . .                                      | 9         |
| 5.3.3    | Multi-signal classification cascade . . . . .                      | 10        |
| 5.3.4    | Usage . . . . .                                                    | 10        |
| 5.4      | melss_labeler.py — GUI Labeler . . . . .                           | 11        |
| 5.4.1    | What it does . . . . .                                             | 11        |
| 5.4.2    | Key features . . . . .                                             | 11        |
| 5.4.3    | Label file format . . . . .                                        | 11        |
| 5.4.4    | Usage . . . . .                                                    | 11        |
| 5.5      | optical_inspection_system.py — Monolithic GUI (OIS v7.3) . . . . . | 12        |
| 5.5.1    | What it does . . . . .                                             | 12        |
| 5.5.2    | Tabs . . . . .                                                     | 12        |
| 5.5.3    | Usage . . . . .                                                    | 12        |
| <b>6</b> | <b>File Reference — train/</b>                                     | <b>12</b> |
| 6.1      | Overview: Why Four Stages? . . . . .                               | 12        |
| 6.2      | augment_dataset.py — Stage 1: Offline Augmentation . . . . .       | 13        |

|          |                                                                 |           |
|----------|-----------------------------------------------------------------|-----------|
| 6.2.1    | What it does                                                    | 13        |
| 6.2.2    | Augmentation operations                                         | 13        |
| 6.2.3    | Output structure                                                | 13        |
| 6.2.4    | Usage                                                           | 13        |
| 6.3      | <code>tile_dataset.py</code> — Stage 3: High-Resolution Tiling  | 13        |
| 6.3.1    | Why tiling matters                                              | 13        |
| 6.3.2    | Tiling strategy                                                 | 14        |
| 6.3.3    | Usage                                                           | 14        |
| 6.4      | <code>train_finetune.py</code> — Stage 4: Two-Phase Fine-Tuning | 14        |
| 6.4.1    | Two-phase strategy                                              | 14        |
| 6.4.2    | Key hyperparameter decisions (v4)                               | 14        |
| 6.4.3    | Usage                                                           | 15        |
| 6.5      | <code>pipeline.py</code> — All Four Stages Orchestrated         | 15        |
| 6.5.1    | What it does                                                    | 15        |
| 6.5.2    | Usage                                                           | 15        |
| <b>7</b> | <b>File Reference — <code>modular_aoi/ois/</code></b>           | <b>16</b> |
| 7.1      | Overview                                                        | 16        |
| 7.2      | <code>utils.py</code> — Data Layer and Shared Constants         | 16        |
| 7.3      | <code>filters.py</code> — Image Processing Filter System        | 17        |
| 7.4      | <code>aoi_engine.py</code> — Core AOI Algorithms v3.9           | 17        |
| 7.5      | <code>threads.py</code> — Background QThread Workers            | 18        |
| 7.6      | <code>theme.py</code> — Visual Theme                            | 18        |
| 7.7      | <code>widgets.py</code> — Custom Qt Widgets                     | 18        |
| 7.8      | <code>main_window.py</code> — Application Entry Point           | 19        |
| 7.9      | <code>tabs/</code> — GUI Tab Implementations                    | 19        |
| <b>8</b> | <b>Data &amp; Assets</b>                                        | <b>20</b> |
| 8.1      | <code>data/Main_dataset/</code>                                 | 20        |
| 8.2      | <code>data/Raw_pics/</code>                                     | 20        |
| 8.3      | <code>data/Real_images/TEST/</code>                             | 20        |
| 8.4      | <code>test_defects/</code>                                      | 20        |
| <b>9</b> | <b>Typical Workflows</b>                                        | <b>21</b> |
| 9.1      | Workflow 1: Train a New Model                                   | 21        |
| 9.2      | Workflow 2: Generate Synthetic Defects and Evaluate Accuracy    | 21        |
| 9.3      | Workflow 3: Inspect Test Boards Against a Golden Reference      | 22        |
| 9.4      | Workflow 4: Live Camera Inspection with the GUI                 | 22        |

## 1 What This Project Does

The MELSS AOI-Pipeline is a complete, production-oriented system for **Automated Optical Inspection (AOI)** of printed circuit boards (PCBs). Given a photograph of a known-good “golden” PCB and photographs of production test boards, the pipeline detects every component on each board, compares them to the golden reference, and classifies any anomaly into one of four defect categories:

### MISSING (MISS)

A component slot that is present in the golden reference has no detectable component in the test board. The injector fills the slot with colour-matched PCB background noise using cosine-feathered Gaussian fill.

### WRONG COMPONENT (WCOM)

A component is present at the correct location but it is the wrong type — e.g. a transistor fitted where a diode should be. The injector swaps in a component from a compatible size group.

### WRONG POLARITY (WPOL)

A polarized component (IC, diode, capacitor, LED, transistor) has been placed with reversed orientation. The injector rotates the patch 180°.

### MISALIGNED (MALI)

A component is present and of the correct type but it is rotated 15–45° away from its correct orientation. The injector uses an elliptical feather mask and `cv2.warpAffine` to blend the rotated patch seamlessly.

The pipeline has three independent layers:

1. **Training layer** (`train/`) — augment a small hand-labelled PCB dataset, remap class IDs, tile into high-resolution crops, then fine-tune a YOLOv8 model in two phases.
2. **Data generation layer** (`src/generate.defect.py`, `src/aoi_evaluator.py`) — synthesise hundreds of labelled defective board images from a single golden photograph; auto-evaluate the detection algorithms against ground truth.
3. **Inspection layer** (`src/pcb_aoi.py`, `src/optical_inspection_system.py`, `modular_aoi/`) — compare a live or offline test board against the golden reference and produce annotated result images and JSON reports.

## 2 Repository Layout

```
AOI-Pipeline/
+-- models/
|   |-- best.pt                               # YOLOv8 model weights (20 MB)
|
+-- data/                                     # Training datasets and raw images
|   +-- Main_dataset/                         # 23-image labelled dataset
|       (images/ + labels/ + data.yaml)
|   +-- Raw pics/                             # 28 original unlabelled source
|       photos
|   |-- Real_images/TEST/                     # 7 real DSLR boards (unlabelled
|       held-out set)
|
+-- test_defects/                           # 31 pre-generated synthetic
    defective boards
```

```

|   +-- original.jpg           # Golden board used to generate
|   the set
|   +-- board_000.png ... board_019.png # 20 boards (no type label
|   in filename)
|   \-- board_0001_WCOM.png etc.  # 11 boards with defect types in
|   filename
|
+-- src/
|   +-- pcb_aoi.py             # Interactive CLI inspector (629
|   lines)
|   +-- generate_defect.py      # Synthetic defect generator (553
|   lines)
|   +-- aoi_evaluator.py        # Accuracy evaluator v3.9 (1703
|   lines)
|   +-- melss_labeler.py        # GUI labeler (1484 lines)
|   +-- optical_inspection_system.py # Monolithic GUI OIS v7.3 (8755
|   lines)
|   \-- _ois_arrows/           # SVG arrow assets for the GUI
|
+-- train/
|   +-- augment_dataset.py      # Stage 1: offline augmentation
|   (445 lines)
|   +-- tile_dataset.py         # Stage 3: high-res tiling (405
|   lines)
|   +-- train_finetune.py       # Stage 4: two-phase fine-tuning
|   (341 lines)
|   \-- pipeline.py             # All 4 stages orchestrated (888
|   lines)
|
\-- modular_aoi/ois/           # Modular split of
   optical_inspection_system.py
   +-- aoi_engine.py            # Core AOI algorithms v3.9 (1429
   lines)
   +-- filters.py               # Filter nodes + auto-calibration
   (617 lines)
   +-- main_window.py           # GUI entry point (567 lines)
   +-- theme.py                 # Colours, fonts, widget factories
   (380 lines)
   +-- threads.py               # Background QThreads (418 lines)
   +-- utils.py                 # Data layer, DB, class list (873
   lines)
   +-- widgets.py               # Custom Qt widgets (6121 lines)
   \-- tabs/                    # One file per GUI tab
       +-- run_tab.py           # Live camera inspection (1201
       lines)
       +-- train_tab.py         # Golden + dataset management (613
       lines)
       +-- filters_tab.py       # Filter pipeline designer (559
       lines)
       +-- aoi_tab.py           # Offline batch AOI (997 lines)
       +-- history_tab.py       # SQLite history viewer (443 lines)
       \-- infer_tab.py         # Single-image inference
           (compatibility wrapper)

```

### 3 Setup

### 3.1 Python Version

Python 3.9 or later is required. PySide6 requires  $\geq 3.8$ . All scripts auto-install missing dependencies via pip on first run (see the `_pip()` helper in `aoi_evaluator.py` and `generate_defect.py`).

### 3.2 Install All Dependencies

```
pip install ultralytics opencv-python numpy PySide6 Pillow scipy \
        pyyaml platformdirs psutil pynvml pytesseract pyzbar sahi
```

What each package is used for:

- `ultralytics` — YOLOv8 model for component detection and training
- `opencv-python` — image I/O, ORB feature matching, homography, all pixel-level operations (CLAHE, Gaussian blur, `warpAffine`, etc.)
- `numpy` — array math throughout (NCC, SSIM, histogram operations)
- `PySide6` — Qt GUI framework for the labeler, OIS GUI, and all tabs
- `Pillow` — image thumbnail generation in the labeler
- `scipy` — `linear_sum_assignment` (Hungarian matching) in the evaluator; gracefully falls back to greedy if unavailable
- `pyyaml` — reading and writing `data.yaml` files
- `platformdirs` — OS-appropriate data directory resolution (`~/.ois_data` fallback on all platforms)
- `psutil` / `pynvml` — real-time CPU/GPU status bar in the GUI
- `pytesseract` — OCR filter node in the filter pipeline
- `pyzbar` — barcode and QR-code scanning filter node
- `sahi` — tiled inference for very large PCB images (longest edge  $> 1920$  px in the GUI)

### 3.3 Model Weights

All scripts resolve the model path relative to the repository root: `models/best.pt`. The repository includes the trained weights (20 MB). To retrain from scratch, use `train/pipeline.py`.

## 4 The 25 Component Classes

The pipeline recognises exactly 25 component classes. Class index order is fixed and must match the `data.yaml` written by the training scripts.

| # | Class name      | #  | Class name          | #  | Class name        |
|---|-----------------|----|---------------------|----|-------------------|
| 0 | Connector (P)   | 9  | Resistor Array (RA) | 18 | Test Point (TP)   |
| 1 | Resistor (R)    | 10 | Resistor Net (RN)   | 19 | LED               |
| 2 | Transformer (T) | 11 | Crystal (CR)        | 20 | Transistor (QA)   |
| 3 | Diode (D)       | 12 | IC (IC)             | 21 | Cap Array (CRA)   |
| 4 | Capacitor (C)   | 13 | Jumper (JP)         | 22 | Motor (M)         |
| 5 | Transistor (Q)  | 14 | Varistor (V)        | 23 | Fuse (F)          |
| 6 | Jumper (J)      | 15 | Button (BTN)        | 24 | Ferrite Bead (FB) |
| 7 | Inductor (L)    | 16 | Switch (SW)         |    |                   |
| 8 | IC (U)          | 17 | Switch (S)          |    |                   |

**Polarized classes** (eligible for WPOL injection and GOH polarity checking): IC (U), IC (IC), Transistor (Q), Transistor (QA), Diode (D), LED, Capacitor (C), Cap Array (CRA).

## 5 File Reference — src/

### 5.1 pcb\_aoi.py — Interactive CLI Inspector

#### *What it does*

The primary command-line inspection tool. Runs YOLO detection on a golden board image to build a reference component map, then for each test board: warps it onto the golden coordinate frame using ORB feature matching and RANSAC homography, runs YOLO detection on the aligned image, and compares the two component sets to produce MISSING, MISALIGNED, WRONG, and EXTRA classifications.

#### *Alignment algorithm*

1. Convert both images to greyscale.
2. Detect ORB keypoints (up to 6000) and compute binary descriptors.
3. Brute-force match descriptors (Hamming distance, crossCheck=True), keep the 300 best by distance.
4. Compute homography  $\mathbf{H}$  via `cv2.findHomography` with RANSAC reprojection threshold 5.0 px.
5. Warp the test image to the golden coordinate frame with `cv2.warpPerspective`. If fewer than 10 matches are found, or homography fails, the raw un-warped image is used and a warning is printed.

#### *Comparison algorithm*

For each golden component  $g$ , the best-IoU test component  $t$  is found.

- $\text{IoU}(g, t) < \text{iou\_match} \Rightarrow \text{MISSING}$
- $\text{IoU} \geq \text{iou\_match}$  but  $t.\text{class} \neq g.\text{class} \Rightarrow \text{WRONG COMPONENT}$
- Same class,  $\|\text{centre}(g) - \text{centre}(t)\|_2 > \text{misalign\_px} \Rightarrow \text{MISALIGNED (offset)}$
- Same class, within centroid tolerance, but rotation angle  $\Delta\theta > \text{misalign\_angle} \Rightarrow \text{MIS-ALIGNED (rotation)}$

Rotation detection uses `_patch_orientation_px`: Canny edges on the component patch  $\rightarrow$  `cv2.minAreaRect` on the largest contour  $\rightarrow$  normalised angle in  $[0^\circ, 180^\circ)$ . Falls back to image moment PCA orientation when no contour is found.

Any test detection not matched to a golden component becomes **EXTRA**.

#### *Default thresholds*

| Constant               | Value                                   |
|------------------------|-----------------------------------------|
| DEFAULT_CONF           | 0.25                                    |
| DEFAULT_IU_MATCH       | 0.30 (min IoU for same-slot match)      |
| DEFAULT_MISALIGN_PX    | 15 px (centroid offset threshold)       |
| DEFAULT_MISALIGN_ANGLE | 8° (rotation threshold)                 |
| DEFAULT_WRONG_IU       | 0.40 (tighter IoU for wrong-class flag) |

#### *Three run modes*

**Mode 1** Detect the golden board and save `golden_reference.json` + annotated image. No test boards checked.

**Mode 2** Load a previously saved `golden_reference.json` and process a batch of test board paths entered at the prompt.

**Mode 3 (default)** Detect golden, then immediately check test boards in the same session.

## Outputs

- `golden_reference.json` — list of `{class, box, conf}` dicts for every component on the golden board
- `golden_annotated.jpg` — golden board with green boxes + class labels
- `golden_raw.jpg` — unmodified golden image (used as alignment target)
- `<board>_aligned.jpg` — test board after homography warp
- `<board>_result.jpg` — annotated image with coloured boxes and HUD: red=MISSING, orange=MISALIGNED, magenta=WRONG, cyan=EXTRA, green=OK; arrowed lines connect misaligned golden and test centroids; top-left HUD shows counts; bottom-right legend; large PASS/FAIL stamp
- `<board>_report.json` — full per-board JSON report with summary counts and detailed defect lists
- `batch_summary.json` — all board reports in one file

## Usage

```
python src/pcb_aoi.py
# Example prompt responses:
#   YOLO model path      -> models/best.pt
#   Output folder        -> /path/to/aoi_output
#   Detection confidence -> 0.25
#   Component match IoU  -> 0.30
#   Misalign threshold px -> 15
#   Misalign angle threshold -> 8
#   Mode                 -> 3
#   Golden board image   -> /path/to/golden_board.jpg
#   Test image path      -> /path/to/board_001.jpg
#   Test image path      -> (empty line to finish)
```

## 5.2 generate\_defect.py — Synthetic Defect Generator

### What it does

Creates a labelled dataset of synthetic defective PCB images from a single golden board photo. All defect injection is pixel-only — the output `images/` folder contains clean board images with no annotations drawn, making them suitable as direct input to the AOI evaluator.

### The four injectors

#### inject\_missing

1. Build a background mask covering all PCB regions *not* occupied by components (with a 10% expansion border around each component).
2. Sample pixel colour statistics (median + std) from background pixels in expanding rings ( $3\times$ ,  $8\times$ ,  $40\times$  the component size) until at least 64 background pixels are collected.
3. Fill the component slot with Gaussian noise sampled from those statistics.
4. Apply a cosine-feathered edge ramp (up to 4 px wide) so the fill blends seamlessly into the surrounding PCB.

#### inject\_wrong\_component

1. Group all components by size class: passive (resistors, capacitors, ferrites, inductors), active-small (transistors, diodes), and ICs by area quartile (`ic_0` through `ic_3`).
2. Find donor candidates in the same group with area ratio  $\leq 4\times$  the victim, confidence  $> 0.15$ , and a different label.
3. Pick the closest-area donor from the top half of candidates by ratio.

4. Resize the donor patch to the victim slot dimensions and paste it in. If the area ratio  $> 1.6$ , apply a small random offset ( $\pm 20\%$  of slot size) to simulate placement imprecision.
5. Validate: if the pasted region is visually indistinguishable from the original (`absdiff.mean() < 8`), fall back to `inject_missing`.

### `inject_wrong_polarity`

1. Only applies to polarized component types.
2. Rotate the component patch exactly  $180^\circ$  using `cv2.ROTATE_180`.
3. Validate: if the rotated patch is symmetric (`absdiff.mean() < 8`), skip (component is visually indistinguishable under rotation).

### `inject_misaligned`

1. Choose a random angle in  $[15^\circ, 45^\circ]$  with random sign.
2. Build a rotation matrix centred on the patch and warp the patch with `cv2.warpAffine` using `BORDER_REPLICATE`.
3. Create a soft elliptical blend mask: erode a binary rotation mask with a  $5 \times 5$  elliptical kernel, then Gaussian blur; blend rotated and original patches using this 3-channel float mask.
4. Validate: if `absdiff.mean() < 5`, the component is too small or symmetric to show rotation; skip.

### *Defect mix and naming*

| Type            | Weight | Abbreviation |
|-----------------|--------|--------------|
| missing         | 0.35   | MISS         |
| wrong-component | 0.30   | WCOM         |
| wrong-polarity  | 0.20   | WPOL         |
| misaligned      | 0.15   | MALI         |

Output filenames encode all injected types, e.g. `board_0006_MALI_WCOM_WPOL.png` contains misaligned, wrong-component, and wrong-polarity defects on that board.

### *Outputs*

- `<out_dir>/images/<stem>.png` — clean defected image, **no** boxes or text drawn, lossless PNG
- `<out_dir>/annotated/<stem>.png` — same image with coloured bounding boxes drawn over every changed slot
- `<out_dir>/annotated/<stem>.json` — per-board ground truth: `[{component_id, original_label, injected_type, ...}]` (only written with `--save_json`)
- Terminal summary table: per-type injection counts and percentages with ASCII bar chart

### *Usage*

```
# Default: 10 boards, up to 5 defects each
python src/generate_defect.py \
    --golden /path/to/golden_board.jpg \
    --model  models/best.pt

# 20 boards, up to 7 defects, JSON sidecars, verbose per-board log
python src/generate_defect.py \
    --golden /path/to/golden_board.jpg \
    --model  models/best.pt \
    --boards 20 \
    --defects 7 \
```



```

--out_dir      generated_defects \
--save_json \
--verbose

# Generate only misaligned defects for targeted evaluation
python src/generate_defect.py \
--golden       /path/to/golden_board.jpg \
--only_defect  misaligned \
--boards       30

```

| Flag          | Default          | Description                                                                   |
|---------------|------------------|-------------------------------------------------------------------------------|
| --golden      | (hardcoded path) | Path to the golden board image                                                |
| --model       | models/best.pt   | Path to YOLO .pt model                                                        |
| --boards      | 10               | Number of defective boards to generate                                        |
| --defects     | 5                | Max defects per board (actual count is random 1...N)                          |
| --conf        | 0.15             | YOLO detection confidence threshold                                           |
| --seed        | 42               | RNG seed for reproducibility                                                  |
| --out_dir     | defect_output    | Root output folder                                                            |
| --only_defect | None             | Restrict to one type: missing, wrong_component, wrong_polarity, or misaligned |
| --verbose     | off              | Print one summary line per board                                              |
| --save_json   | off              | Write JSON sidecar files into annotated/                                      |

### 5.3 aoievaluator.py — Accuracy Evaluator v3.9

#### What it does

A self-contained regression testing tool. For each of --boards synthetic test boards it:

1. Injects defects into a copy of the golden image (same injectors as `generate_defect.py`).
2. Runs the full AOI inference engine (same algorithm as `aoi_engine.py`) on the defected image.
3. Compares detected defects against injected ground truth and records True Positives, False Positives, and False Negatives per defect type.

Final output: per-type Precision / Recall / F1 and a  $4 \times 4$  confusion matrix showing how each injected type was classified.

#### AOI inference zones

The inference engine `_aoi_check_board` classifies each golden component slot into one of three zones based on the pixel-level region difference score `rdiff` (mean absolute pixel difference in the component region between golden and test, after Gaussian blur):

**Zone A** (`rdiff < low_thr`) The slot looks unchanged. Skipped unless the component is polarized, in which case the NCC polarity check is run to catch subtle wrong-polarity defects that do not create significant pixel differences.

**Zone B** (`low_thr ≤ rdiff < high_thr`) Moderate change. Could be misalignment, wrong component, or wrong polarity. Runs the full multi-signal cascade: misalignment sweep first, then cross-detection ownership check, then offset misalignment, then rotation misalignment, then polarity analysis.

**Zone C** (`rdiff ≥ high_thr`) Large change. Runs same cascade but with higher confidence in defect presence. Includes the variance and edge-density gates before emitting “missing”.

`low_thr` and `high_thr` are per-component adaptive thresholds computed by `_aoi_calibrate`: it

runs 2 brightness-jitter passes on the golden image and measures the natural variation at each slot.

### *Multi-signal classification cascade*

**1. Two-pass misalignment check (Fix P)** Applies a rotation recovery NCC sweep over the component patch:

- **Coarse pass:** angles  $-45^\circ$  to  $+45^\circ$  in  $5^\circ$  steps. Records the best recovered NCC and angle.
- **Fine pass:**  $1^\circ$  steps over  $[\text{coarse\_best} \pm 8^\circ]$ . Closes the gap at boundary angles ( $10^\circ$ – $20^\circ$ ) where coarse sampling undersamples the NCC peak.

Classified as MISALIGNED if:  $\text{best\_ncc} \geq 0.60$  **and**  $\text{gain} = \text{best\_ncc} - \text{unrotated\_ncc} \geq 0.06$  **and**  $|\text{best\_angle}| \geq 10^\circ$ .

**2. Cross-detection ownership** If a detection of a *different* class is closer to this golden slot than any other golden slot of its own class, it is classified as WRONG COMPONENT regardless of the pixel difference.

**3. Offset misalignment** If the same-class detection centroid is displaced  $> \text{detect\_tol} \times 1.6$  from the golden centroid, classified as MISALIGNED (type: offset).

**4. NCC polarity check (baseline)** Computes ZNCC between the golden patch and the test patch, then compares with ZNCC after rotating the golden patch  $90^\circ$ ,  $180^\circ$ , and  $270^\circ$ . If the  $180^\circ$  rotation gives a higher correlation by  $> 0.05$  for regular components or  $> 0.28$  for ICs, classified as WRONG POLARITY.

**5. GOH polarity check (Fix N)** Builds 12-bin Gradient Orientation Histograms ( $30^\circ/\text{bin}$  over  $[-\pi, +\pi]$ ) from Sobel gradients of the golden and test patches. A  $180^\circ$  rotation shifts the histogram by exactly 6 bins. If  $\text{NCC}(\text{rolled golden histogram}, \text{test histogram}) - \text{NCC}(\text{golden histogram}, \text{test histogram}) \geq 0.08$ , classified as WRONG POLARITY. In the uncertain band  $[0.05, 0.10]$ , the intensity asymmetry opposition score (top/bottom and left/right asymmetry sign comparison) casts a tie-breaking vote.

**6. Edge-density rescue (Fix O)** Before emitting “missing” in Zone C when variance is low, Canny edges are computed on the test patch (low thresholds 20/60 to catch faint resized donors). If edge density  $\geq 0.045$  (Gaussian fill: 0.00–0.01; real donor: 0.04–0.18), classified as WRONG COMPONENT rather than MISSING.

**7. SSIM and template matching** Structural similarity and normalised cross-correlation template scores are used in various Zone B/C branches as confirmation gates for WCOM detection.

### *Usage*

```
# Default 100-board run
python src/aoi_evaluator.py \
    --golden /path/to/golden_board.jpg \
    --model  models/best.pt

# Targeted misalignment evaluation with full trace
python src/aoi_evaluator.py \
    --golden      /path/to/golden_board.jpg \
    --only_defect misaligned \
```

```
--boards      50 \
--trace \
--save_viz    mali_debug.png
```

| Flag          | Default        | Description                                               |
|---------------|----------------|-----------------------------------------------------------|
| --golden      | required       | Path to golden board image                                |
| --model       | models/best.pt | Path to YOLO .pt model                                    |
| --boards      | 100            | Number of synthetic test boards                           |
| --defects     | 5              | Max defects per board                                     |
| --conf        | 0.15           | YOLO confidence threshold                                 |
| --seed        | 42             | RNG seed                                                  |
| --verbose     | off            | Print per-board injected vs. detected                     |
| --debug       | off            | Print SSIM/NCC/GOH signals for failed slots               |
| --trace       | off            | Print diagnostics for all Zone B/C slots on failed boards |
| --save_viz    | ""             | Save annotated debug image (PNG)                          |
| --only_defect | None           | Restrict to one defect type                               |

## 5.4 melss\_labeler.py — GUI Labeler

### What it does

A standalone PySide6 application for creating and editing YOLO-format bounding-box label files. Used to hand-annotate raw PCB photographs before feeding them into the training pipeline.

### Key features

- **Drawing:** click-drag to draw a box; click an existing box to select and resize; Delete key or right-click context menu to remove.
- **Navigation:** scroll-wheel to zoom toward cursor; middle-mouse-button drag to pan when zoomed; arrow keys to step through the image list.
- **Auto-save:** label .txt file is written to disk on every add, edit, or delete — no explicit save button needed.
- **Create Project:** scaffolds images/, labels/, and data.yaml in a new directory with the full 25-class list.
- **Auto-label:** runs YOLO inference (models/best.pt) on every unlabelled image in the project and pre-populates .txt files; runs in a background thread so the GUI stays responsive.
- **Class selector:** dropdown with all 25 classes; keyboard shortcuts for the four most common: Q (Transistor), R (Resistor), C (Capacitor), U (IC).
- **Thumbnail strip:** scrollable sidebar with all images; labelled images show a green dot; images with 0 boxes show red.

### Label file format

Each <image\_stem>.txt contains one YOLO-format line per component:

```
<class_id> <cx> <cy> <width> <height>
```

All coordinates are normalised to [0,1] relative to image dimensions.

### Usage

```
python src/melss_labeler.py
# File -> Open Project (select a folder with images/ and labels/
#                        subfolders)
# or use "Create Project" to start fresh
```

No command-line arguments.

## 5.5 optical\_inspection\_system.py — Monolithic GUI (OIS v7.3)

### *What it does*

The original single-file PySide6 application for the MELSS Optical Inspection System. At 8755 lines it is the largest file in the project and contains everything the modular package provides, in one file. Kept for reference and for environments where a single-file deployment is preferred.

### *Tabs*

#### **Run**

Opens a camera feed (auto-selects backend: `CAP_DSHOW` on Windows, `CAP_AVFOUNDATION` on macOS, `CAP_V4L2` on Linux). Streams at 30 fps display / 15 fps inference. The user captures a golden frame, then the system compares every subsequent frame against the golden using both pixel diff and the full AOI v3.9 engine. A large **PASS/FAIL** annunciator updates each frame.

#### **Golden / Train**

Manages the golden reference image and the labelled dataset. Buttons: capture golden from live camera, load golden from file, run `AutoLabelThread` to pre-label all project images, run `AugThread` for background augmentation, and trigger `TrainingThread` for a full two-phase fine-tune. Progress bar and live log streamed from the training subprocess.

#### **Filters / Logic**

Visual pipeline designer. Add, remove, reorder, and configure filter nodes. A live preview shows the effect of the pipeline on the most recent camera frame. The `AutoCalibrateWorker` button scores all filter permutations against the golden and recommends the highest-scoring combination.

#### **AOI (Offline)**

Batch inspect a folder of images against the golden without a live camera. Results are shown in a scrollable strip; each entry shows a thumbnail, PASS/FAIL status, and defect counts. Export saves all results as JSON and annotated JPEG images.

#### **Infer**

Single-image or folder YOLO inference viewer. Drag-drop or browse to load an image; detections shown with class labels and confidence scores.

#### **History**

SQLite-backed inspection log (WAL journal mode for concurrent access). Filterable by date, board ID, and PASS/FAIL status. Export to CSV.

### *Usage*

```
python src/optical_inspection_system.py
```

No command-line arguments. Data directory is resolved via `platformdirs` (typically `~/.ois_data`).

## 6 File Reference — train/

### 6.1 Overview: Why Four Stages?

A typical PCB AOI dataset starts with only a handful of labelled photographs. Training a YOLO model directly on 23 images would result in severe overfitting. The four-stage pipeline addresses this systematically:

1. **Augment** — expand 23 images to 280 using PCB-appropriate transforms (no heavy warps

- that would make labels inaccurate).
2. **Remap** — scan the augmented labels to find which of the 25 class IDs actually appear; compact them to a dense 0-based range so the model head has no wasted outputs.
  3. **Tile** — slice the full-resolution originals into 960 px crops with 25% overlap, so each small component (resistor, cap) appears at 40+ px width instead of 6 px after downscaling.
  4. **Train** — fine-tune from the existing `best.pt` in two phases: frozen backbone warm-up, then full unfreezing.

## 6.2 `augment_dataset.py` — Stage 1: Offline Augmentation

### *What it does*

Expands a YOLO-format labelled dataset to a target size using a configurable set of transforms. Every spatial transform correctly remaps bounding boxes. Boxes that lose more than 70% of their area after clipping, or that shrink below 0.02% of image area, are dropped.

### *Augmentation operations*

#### **Spatial (applied first, boxes updated)**

Horizontal flip (50% probability), optional vertical flip (off by default — PCBs are rarely upside-down), random 90°/180°/270° rotation (40%), small affine: rotation  $\pm 5^\circ$  + scale 0.85–1.15 $\times$  + translation  $\pm 8\%$  (always applied), perspective warp strength 0.04 (20%).

#### **Pixel-level (applied after spatial)**

Brightness 0.70–1.35 $\times$ , contrast 0.75–1.30 $\times$ , saturation 0.70–1.40 $\times$  (in HSV space), hue shift  $\pm 10^\circ$ , gamma 0.75–1.40.

#### **Noise / blur**

Gaussian noise variance 5–25 (40% probability), motion blur kernel 3–7 px (20%), median blur kernel 3–5 px (15%).

#### **Compression simulation**

JPEG quality 60–95 (30%), mild grid distortion 5-step barrel/pin-cushion with limit 0.05 (15%; boxes are not updated for this — distortion is too mild to matter).

### *Output structure*

The augmented images are staged, shuffled, then split 80/20 into `train/` and `val/` subdirectories. A `data.yaml` with the full 25-class list is written automatically.

### *Usage*

```
python train/augment_dataset.py \
  --src      /path/to/labelled_project \
  --dst      /path/to/augmented_output \
  --target 280 \
  --seed    42
```

| Flag                  | Default     | Description                                                            |
|-----------------------|-------------|------------------------------------------------------------------------|
| <code>--src</code>    | (hardcoded) | Source folder containing <code>images/</code> and <code>labels/</code> |
| <code>--dst</code>    | (hardcoded) | Output directory                                                       |
| <code>--target</code> | 280         | Desired total image count (originals + generated)                      |
| <code>--seed</code>   | 42          | Random seed                                                            |

## 6.3 `tile_dataset.py` — Stage 3: High-Resolution Tiling

### *Why tiling matters*

A 4000  $\times$  3000 px PCB photo downscaled to 960 px for training compresses a 2 mm resistor to about 6 px wide — too small for the model to learn reliable features. Tiling extracts that same

resistor at  $\approx 40$  px wide in a 960 px tile, matching its resolution at inference time when the full image is processed tile-by-tile.

### Tiling strategy

1. Prefer the **original full-resolution images** as the tile source (specified via `--src`). Falls back to the augmented set if originals are not available.
2. For each image, slide a  $960 \times 960$  px window with 25% overlap (= 240 px step), covering the entire image.
3. For each tile, clip all YOLO boxes to the tile boundary and remap coordinates to tile-local normalised space.
4. Drop boxes with  $< 25\%$  area remaining after clipping or  $< 16 \text{ px}^2$  absolute area.
5. Keep 5% of empty (background-only) tiles to teach the model what bare PCB looks like.
6. The val/ split is taken from the augmented set untiled (to avoid leakage of high-res originals into validation).

### Usage

```
python train/tile_dataset.py \
  --src /path/to/original_project \
  --aug /path/to/augmented_output \
  --dst /path/to/tiled_output \
  --tile 960 \
  --overlap 0.25 \
  --bg-keep 0.05
```

| Flag                      | Default  | Description                                         |
|---------------------------|----------|-----------------------------------------------------|
| <code>--src</code>        | required | Source dataset (originals preferred)                |
| <code>--aug</code>        | None     | Augmented set dir (for class names and val split)   |
| <code>--dst</code>        | required | Output tiled dataset directory                      |
| <code>--tile</code>       | 960      | Tile size in pixels                                 |
| <code>--overlap</code>    | 0.25     | Overlap fraction between adjacent tiles             |
| <code>--bg-keep</code>    | 0.05     | Fraction of empty (background-only) tiles to keep   |
| <code>--min-vis</code>    | 0.25     | Minimum box visibility fraction after tile clipping |
| <code>--min-area</code>   | 16       | Minimum box area ( $\text{px}^2$ ) after clipping   |
| <code>--no-aug-val</code> | off      | Also tile the val set (default: copy val untiled)   |
| <code>--seed</code>       | 42       | Random seed                                         |

## 6.4 train\_finetune.py — Stage 4: Two-Phase Fine-Tuning

### Two-phase strategy

**Phase 1 — Frozen backbone warm-up (10 epochs)** Only the YOLO detection head is trained. The backbone weights are frozen (`freeze=10`). This prevents the head (which starts with random weights after class remapping) from corrupting the backbone’s learned features with large early gradients. Early stopping is disabled for this phase.

**Phase 2 — Full fine-tune (remaining epochs)** All layers are unfrozen. AdamW optimiser, cosine learning rate schedule decaying from `lr0=0.0005` to `lr0*lr0f=2.5e-5`. Mixed precision (AMP) enabled. Early stopping with `patience=20`. After training, the best weights are exported to ONNX (opset 17, simplified) and TorchScript.

### Key hyperparameter decisions (v4)

- `lr0=0.0005`: halved from v3’s 0.001 to fix NaN class-loss spikes at epoch 3 caused by LR instability.

- mosaic=0.4: tiled images *are* small crops; mosaic teaches the model to assemble context from adjacent tiles at inference time.
- iou=0.60: previous threshold of 0.70 over-suppressed valid predictions on small tiled components (recall=24%); lower IoU increases positive assignments during training.
- scale=0.20: large scale jitter ( $\pm 30\%$ ) distorts small components in 960 px tiles;  $\pm 20\%$  is safer.
- imgsz=960: matches the tile size exactly — no double resize.

### Usage

```
python train/train_finetune.py \
  --data /path/to/tiled_output/data.yaml \
  --model models/best.pt \
  --name my_run_v2 \
  --epochs 120 \
  --imgsz 960 \
  --batch 4 \
  --device 0
```

| Flag     | Default             | Description                 |
|----------|---------------------|-----------------------------|
| --data   | (hardcoded)         | Path to data.yaml           |
| --model  | models/best.pt      | Base .pt to warm-start from |
| --name   | gatekeeper_v4_tiled | YOLO run name               |
| --epochs | 120                 | Total epochs                |
| --imgsz  | 960                 | Training image size         |
| --batch  | 4                   | Batch size (-1 for auto)    |
| --device | 0                   | Device: 0, cpu, 0,1         |

## 6.5 pipeline.py — All Four Stages Orchestrated

### What it does

Runs all four training stages end-to-end from a single command. Also implements **Stage 2 (Class ID Remap)** internally, which is not exposed as a standalone script:

1. Scan every .txt label file in the augmented dataset to find which of the 25 class IDs actually appear.
2. Build a compact remapping (e.g. if only IDs 0, 1, 3, 8 are used, remap to 0, 1, 2, 3).
3. Rewrite all label files and data.yaml in-place.

This prevents the model from wasting capacity on classes never seen in the training data.

### Usage

```
# Full run
python train/pipeline.py \
  --src /path/to/raw_project \
  --dst-aug /path/to/augmented_output \
  --dst-tile /path/to/tiled_output \
  --model models/best.pt \
  --name my_run_v2 \
  --epochs 120 --batch 4 --device 0

# Skip stages already completed
python train/pipeline.py \
  --src /path/to/raw_project \
```

```

--dst-aug /path/to/my_aug --dst-tile /path/to/my_tiled \
--skip-augment --skip-remap

# Start from tile stage
python train/pipeline.py \
    --dst-aug /path/to/my_aug --dst-tile /path/to/my_tiled \
    --from-stage tile

# Dry-run the remap (print plan, do not write)
python train/pipeline.py \
    --dst-aug /path/to/my_aug \
    --from-stage remap --dry-run-remap

```

| Flag            | Default             | Description                                |
|-----------------|---------------------|--------------------------------------------|
| --src           | (hardcoded)         | Raw labelled dataset                       |
| --dst-aug       | (hardcoded)         | Augmented output directory                 |
| --dst-tile      | (hardcoded)         | Tiled output directory                     |
| --model         | models/best.pt      | Base model                                 |
| --name          | gatekeeper.v4_tiled | Run name                                   |
| --from-stage    | —                   | Start from: augment, remap, tile, or train |
| --skip-augment  | off                 | Skip Stage 1                               |
| --skip-remap    | off                 | Skip Stage 2                               |
| --skip-tile     | off                 | Skip Stage 3                               |
| --skip-train    | off                 | Skip Stage 4                               |
| --dry-run-remap | off                 | Print remap plan without writing           |
| --target        | 280                 | Augmentation target count                  |
| --tile          | 960                 | Tile size (px)                             |
| --overlap       | 0.25                | Tile overlap                               |
| --epochs        | 120                 | Training epochs                            |
| --imgsz         | 960                 | Training image size                        |
| --batch         | 4                   | Batch size                                 |
| --device        | 0                   | Training device                            |

## 7 File Reference — modular\_aoi/ois/

### 7.1 Overview

The `modular_aoi/ois/` package is a clean refactoring of `optical_inspection_system.py` into focused modules. All functionality is identical; the split makes each concern independently readable and testable. Entry point: `python modular_aoi/ois/main_window.py`.

### 7.2 `utils.py` — Data Layer and Shared Constants

Pure data layer with no Qt widgets. Contains:

- **Feature flags:** `HAS_CV2`, `HAS_NP`, `HAS_YOLO`, `HAS_SAH`, `HAS_OCR`, `HAS_ZBAR` — checked at import time; all features degrade gracefully if a dependency is missing.
- `DATA_ROOT`: resolved via `platformdirs.user_data_dir` with fallback to `~/.ois_data`. All session data, history, and project configs are stored here.
- `AOIComp` **dataclass**: `id`, `label`, `conf`, `cx`, `cy`, `w`, `h` (normalised coordinates). All AOI engine functions use this type.
- `MASTER_CLASS_LIST`: the authoritative ordered list of all 25 class names. Index position = YOLO class ID.



- **HistoryDB:** SQLite database with WAL journal mode. Schema: `inspections(id, timestamp, board_id, status, defect_json)`. Methods: `insert`, `query(filters)`, `export_csv`.
- **ProjectConfig:** JSON-backed project settings (model path, confidence, filter pipeline, golden image path, output directory). Auto-saves on every change.
- **match\_detections:** simple IoU-based matching between two component lists; returns matched pairs and unmatched indices.
- **Model helpers:** `load_optimized_yolo` (loads and optionally fuses model for inference speed), `find_best_pt` (searches the project and `models/` directory for the best available weights), `safe_predict` (wraps `model.predict` with exception handling).
- **DEFAULT\_AUG:** augmentation configuration dict shared by the training tab and `AugThread`.

### 7.3 filters.py — Image Processing Filter System

Implements a node-based image pre-processing pipeline that sits between the camera feed and the AOI engine. Each filter is a `FilterNode` subclass with an `apply(img)` method and a `params` dict for GUI-exposed parameters.

#### Available filter nodes:

- **CLAHEFilter** — Contrast Limited Adaptive Histogram Equalisation in LAB colour space (params: `clip_limit`, `tile_size`). CLAHE instance is cached and re-created only when parameters change.
- **GaussBlurFilter** — Gaussian blur (param: `kernel`, auto-forced to odd).
- **CannyFilter** — Canny edge detection (`thresh1`, `thresh2`).
- **AdaptThreshFilter** — Adaptive threshold (Gaussian, `block_size`, `c`).
- **SharpenFilter** — Unsharp mask sharpening.
- **GammaFilter** — Gamma correction via LUT.
- **ThresholdFilter** — Global binary threshold.
- **BilateralFilter** — Edge-preserving bilateral blur.
- **OCRFilter** — Runs Tesseract OCR on the image and overlays the extracted text (requires `pytesseract`).
- **BarcodeFilter** — Decodes barcodes and QR codes using `pyzbar` and overlays decoded values.
- **ROIFilter** — Masks the image to a user-defined region of interest.

**Auto-calibration:** `AutoCalibrateWorker` is a `QThread` that scores filter pipeline permutations against the golden reference image. The scoring function measures SSIM between the filtered golden and a brightness-jittered copy; the combination with the highest mean SSIM is recommended as the best pipeline for this particular board.

### 7.4 aoi\_engine.py — Core AOI Algorithms v3.9

Contains all detection and comparison logic. Mirrors `src/aoi_evaluator.py` exactly in its decision thresholds and algorithm structure. Key functions:

- `_aoi_infer_arr`: runs YOLO (or SAHI tiled inference for large images) and returns a list of `_AOIComp` objects, after NMS.
- `_aoi_calibrate`: computes per-component adaptive thresholds by running 2 jitter passes on the golden image.
- `_aoi_hungarian_match`: optimal assignment of detections to golden slots using `scipy.linear_sum_assignment` (greedy fallback).
- `_aoi_check_board`: main inspection function; implements the three-zone (A/B/C) multi-signal cascade described in Section 5.3.
- `_aoi_misalignment_check`: two-pass rotation recovery (Fix P).

- `_aoi_goh_polarity`: GOH polarity check (Fix N).
- `_aoi_intensity_asymmetry_score`: tie-breaker asymmetry vote.
- `_aoi_edge_density`: Canny edge density rescue (Fix O).
- `render_overlay`: draws coloured bounding boxes and a PASS/FAIL banner onto a BGR image for display.
- `OfflineAOIThread`: a `QThread` that runs `_aoi_check_board` on a batch of images and emits progress and result signals to the `OfflineAOITab`.

## 7.5 threads.py — Background QThread Workers

All workers inherit `QThread` and communicate via Qt signals to keep the GUI responsive.

### CameraThread

Opens the camera using the OS-appropriate backend. Runs at 30 fps display / 15 fps inference. Display frames are resized to 960 px width; inference frames are stored in a mutex-protected slot and consumed by `InferenceThread`.

### InferenceThread

Polls `CameraThread.get_infer_frame()` in a loop, runs YOLO, and emits the detection list via signal. Runs at 15 fps.

### ThumbThread

Loads thumbnail images from disk asynchronously and emits them for display in the image list sidebar of the training tab.

### TrainingThread

Calls `model.train(...)` in a background thread, capturing the Ultralytics progress callbacks and forwarding them as log-line signals.

### AugThread

Runs the augmentation pipeline from `augment_dataset.py` in the background and emits a completion signal.

### AutoLabelThread

Iterates over all images in the project directory, runs YOLO on each, and writes `.txt` label files. Emits per-image progress signals.

## 7.6 theme.py — Visual Theme

Defines the dark UI colour palette and typography used throughout all tabs. Key constants (all `str` hex colour values or `QFont` objects):

- **Backgrounds**: `BG_DEEP` (darkest), `BG_PANEL`, `BG_CARD`, `BG_CARD2`; `BG_BORDER`, `BG_BORDER2`.
- **Accent colours**: `CYAN` / `CYAN_DIM` / `CYAN_GLOW`, `GREEN`, `AMBER`, `RED`, `PURPLE`.
- **Text**: `TEXT_PRI`, `TEXT_SEC`, `TEXT_DIM`.
- **Fonts**: `_F_MONO_8` through `_F_MONO_12` (monospace at 8–12 pt).
- **Factories**: `make_card(title)` returns a styled `QFrame`; `make_sep()` returns a horizontal separator; `sec_lbl(text)` returns a dimmed `QLabel`; `set_fg(widget, colour)` sets foreground colour inline.

## 7.7 widgets.py — Custom Qt Widgets

### FastLog

High-performance log widget based on `QPlainTextEdit`. Text appended from any thread is buffered and flushed to the UI in 100 ms batches. Caps at 500 lines to prevent memory growth during long runs.

### AnnunciatorBanner

Large PASS/FAIL status display with colour-coded states: ready (grey), setup (amber), pass (green), fail (red). Sized to be visible from a distance on the production floor.

**StatCard**

Small card widget showing a label and a live-updating numeric value. Used in the Run tab for MISSING/MISALIGNED/WRONG counters.

**VideoWidget**

Displays a QImage frame from the camera thread. Scales to widget size maintaining aspect ratio.

**LabelCanvas**

Interactive bounding-box drawing canvas. Handles mouse events for draw/select/resize/delete. Emits `box_added` and `box_removed` signals. Used in the labeler and training tab.

**ResultStrip**

Horizontal scrollable strip of board thumbnails with PASS/FAIL colour overlay. Used in the offline AOI tab.

**InferTab**

Single-image and folder YOLO inference viewer with detection overlay. Re-exported from `tabs/infer_tab.py` as a compatibility wrapper.

**ToastManager**

Non-blocking notification popups in the corner of the window.

**ZoomableImageView**

Scrollable/zoomable image viewer with scroll-to-zoom and middle-mouse-pan. Used in the AOI result detail view.

**ROICanvas / ROIZone**

Interactive ROI zone editor for the filters tab. Draw, resize, and label rectangular regions of interest.

Additionally, `widgets.py` contains a complete copy of all v3.9 AOI engine functions (`_aoi_misalignment.check`, `_aoi_goh.polarity`, `_aoi_edge.density`, etc.) so that the monolithic `optical_inspection_system.py` and the modular package share identical detection behaviour without a cross-file import.

## 7.8 `main_window.py` — Application Entry Point

Defines `Sidebar` (left navigation panel with icon buttons for each tab) and `MainWindow` (`QMainWindow` with `QStackedWidget` holding all six tabs). Sets per-OS thread limits for OpenBLAS/MKL before importing Qt to prevent thread-count conflicts with PyTorch workers.

```
python modular_aoi/ois/main_window.py
```

## 7.9 `tabs/` — GUI Tab Implementations

Each file is a `QWidget` subclass. None have `__main__` blocks.

| File                    | Class               | Description                                                                                                                                                                                                                                                                                                                               |
|-------------------------|---------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>run_tab.py</code> | <code>RunTab</code> | Live camera inspection. Manages <code>CameraThread</code> + <code>InferenceThread</code> . On each frame: applies the filter pipeline, runs YOLO, optionally runs the full AOI engine against the stored golden, updates the <code>AnnunciatorBanner</code> and <code>StatCard</code> counters, saves results to <code>HistoryDB</code> . |

|                             |               |                                                                                                                                                                                                                                                                                                                           |
|-----------------------------|---------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>train_tab.py</code>   | GoldenTab     | Golden reference and dataset management. Capture golden from camera or load from file. Thumbnail grid of all project images with labelled/unlabelled indicator. Buttons to run <code>AutoLabelThread</code> , <code>AugThread</code> , and <code>TrainingThread</code> . Shows live training loss curves via log parsing. |
| <code>filters_tab.py</code> | LogicTab      | Filter pipeline designer. Drag-to-reorder node list; parameter editors for each node; live preview panel; <code>AutoCalibrateWorker</code> controls with progress bar; save/load pipeline to JSON.                                                                                                                        |
| <code>aoi_tab.py</code>     | OfflineAOITab | Offline batch AOI inspection. Browse to a folder; loads golden from <code>ProjectConfig</code> ; runs <code>OfflineAOIThread</code> over all images; shows per-image <code>ResultStrip</code> thumbnails; clicking a result opens <code>ZoomableImageView</code> with annotated overlay; exports JSON and JPEG.           |
| <code>history_tab.py</code> | HistoryTab    | SQLite history viewer. Sortable/filterable table ( <code>QTableWidget</code> ). Filter by date range, status, or board ID text search. Export selected rows to CSV. Double-click to view the full defect JSON for a session.                                                                                              |
| <code>infer_tab.py</code>   | InferTab      | Compatibility re-export of <code>InferTab</code> from <code>widgets.py</code> . Single-image and folder YOLO inference viewer.                                                                                                                                                                                            |

## 8 Data & Assets

### 8.1 data/Main\_dataset/

The primary labelled training dataset. 23 PCB images with YOLO-format `.txt` annotations and a `data.yaml` class map. These are the starting images for `train/augment_dataset.py` (`--src`).

Images come from a mix of publicly available PCB photographs and five real DSLR captures (filenames `Riyan_20260324_...`).

### 8.2 data/Raw pics/

28 original unlabelled source images — the photos before annotation. Includes 5 real DSLR boards. Use these as input to `src/melss_labeler.py` when adding new annotations, or as golden board candidates for `src/generate_defect.py`.

### 8.3 data/Real\_images/TEST/

7 real captured board photos (Riyan DSLR) used as a held-out validation set. The `labels/` subdirectory is empty — these images are run through the inspector to verify real-world detection performance and are not used for training.

### 8.4 test\_defects/

A pre-generated synthetic defect dataset (31 boards total) created with `src/generate_defect.py` from `original.jpg`.

- `original.jpg` — the golden board used to generate this set

- board\_000.png through board\_019.png — 20 original unnamed boards
- board\_0001\_WCOM.png through board\_0010\_WCOM.png — 11 named boards; defect types encoded in filename (MISS, WCOM, WPOL, MALI)

All boards live directly in test\_defects/ (no images/ subfolder). There is no manifest.json — ground truth for named boards is in the filename. To run the evaluator against this set:

```
python src/aoi_evaluator.py \
    --golden test_defects/original.jpg \
    --model models/best.pt \
    --boards 100 \
    --defects 5
```

## 9 Typical Workflows

### 9.1 Workflow 1: Train a New Model

```
# Step 1: Label your PCB images
python src/melss_labeler.py
# Create Project -> add images -> draw boxes -> use Auto-label to
pre-fill

# Step 2: Run all 4 training stages end-to-end
python train/pipeline.py \
    --src /path/to/labelled_project \
    --dst-aug /path/to/my_aug \
    --dst-tile /path/to/my_tiled \
    --model models/best.pt \
    --name my_model_v1 \
    --epochs 120 --imgsz 960 --batch 4 --device 0

# Trained weights: <PROJECT_DIR>/my_model_v1/weights/best.pt
# Copy to models/best.pt for use in all inspection scripts
cp <runs_dir>/my_model_v1/weights/best.pt models/best.pt
```

### 9.2 Workflow 2: Generate Synthetic Defects and Evaluate Accuracy

```
# Step 1: Generate 20 labelled defective boards (all 4 defect types)
python src/generate_defect.py \
    --golden /path/to/golden_board.jpg \
    --model models/best.pt \
    --boards 20 --defects 7 \
    --out_dir generated_defects \
    --save_json --verbose

# Step 2: Run the accuracy evaluator (auto-inject -> detect -> report)
# This generates its OWN boards internally; no need to pass
generated_defects
python src/aoi_evaluator.py \
    --golden /path/to/golden_board.jpg \
    --model models/best.pt \
    --boards 100 --defects 7 \
    --verbose --debug
# Prints per-type Precision/Recall/F1 and a 4x4 confusion matrix
```

### 9.3 Workflow 3: Inspect Test Boards Against a Golden Reference

```
# One-shot: detect golden + check test boards in the same session
# (Mode 3)
python src/pcb_aoi.py
# At prompts:
#   model path           -> models/best.pt
#   output folder        -> /path/to/aoi_output
#   conf                 -> 0.25   iou_match -> 0.30
#   misalign px          -> 15     misalign angle -> 8
#   Mode                 -> 3
#   golden image         -> /path/to/golden_board.jpg
#   test image paths     -> /path/to/board_001.jpg ... (empty to
#   finish)
# Results: <board>_result.jpg, <board>_report.json, batch_summary.json
```

### 9.4 Workflow 4: Live Camera Inspection with the GUI

```
# Modular version (recommended)
python modular_aoi/ois/main_window.py

# Or the monolithic version
python src/optical_inspection_system.py

# In the GUI:
# 1. Run tab -> select camera -> Start Feed
# 2. Capture Golden (aim camera at the known-good board, press button)
# 3. Aim at production test boards -> PASS/FAIL updates each frame
# 4. History tab -> review all session results
```